Assignment #4 - Facade Pattern

Option B: Online Order Processing Facade

Name: Serekpayev Ali

1. Introduction

The project is a simple online order processing system written in Java. To make an order, the system must check product availability, reserve products, process the payment, create a shipment and notify the customer. Each of these steps is done by a separate class: `InventoryService`, `PaymentProcessor`, `ShippingService` and `NotificationService`.

Without a Facade, the client would have to create all these classes, remember the correct order of calls, and write the rollback logic by itself. This makes the client code long and easy to break. For example, if the client forgets to release the stock after a declined payment, the products stay reserved forever.

The Facade Pattern solves this problem. The class `OrderFacade` provides simple methods `placeOrder(cart, customer)` and `cancelOrder(orderId)`. The client calls only them, and the Facade coordinates all services and does the rollback when something fails.

The order of steps inside `placeOrder()` is: (1) check availability, (2) reserve products, (3) process payment, (4) create shipment if the payment succeeded, (5) notify the customer. If the stock check fails or the payment is declined, the Facade stops, releases the reserved products when needed, and sends a failure notification.

2. UML Class Diagram

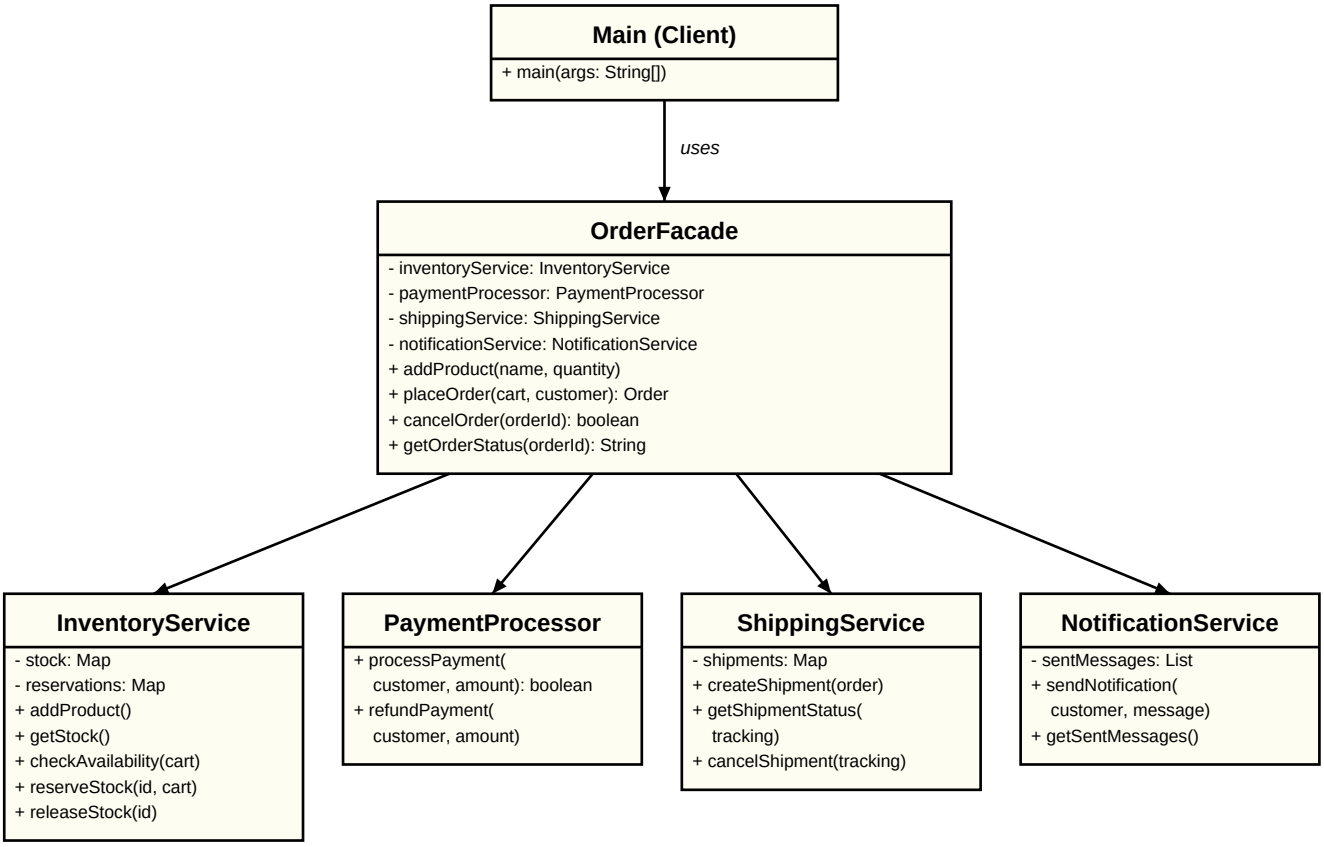

Figure 1. Class diagram: Client, Facade and subsystem classes

The Client (`Main`) knows only `OrderFacade`. The Facade holds and uses the four subsystem classes. Data classes (`Cart`, `CartItem`, `Customer`, `Order`, `OrderStatus`) are left out of the diagram to keep it clear.

3. Clean Code

3.1 Meaningful Names

Names show what the method does, so extra comments are almost not needed. A reader can understand the idea of the program just by reading the method names.

```
public Order placeOrder(Cart cart, Customer customer)   // place an order
public void releaseStock(int orderId)                   // give reserved products back
public boolean checkAvailability(Cart cart)             // check that products are in stock
```

3.2 Single Responsibility Principle

Each class has only one reason to change. `PaymentProcessor` works only with money and does not know anything about stock or delivery.

```
public class PaymentProcessor {
    public boolean processPayment(Customer customer, double amount) { ... }
    public void refundPayment(Customer customer, double amount) { ... }
}
```

3.3 Encapsulation

Fields are private. The services are private fields of the Facade, so the client cannot reach them through `OrderFacade`. The client even fills the warehouse through the Facade method `addProduct()`. Inside `InventoryService` the stock map is also private and is changed only by its own methods.

```
public class OrderFacade {
    private InventoryService inventoryService;
    private PaymentProcessor paymentProcessor;
    private ShippingService shippingService;
    private NotificationService notificationService;
    ...
}
```

3.4 Small, Focused Classes

Inventory, payment, shipping and notification logic are in four separate small classes, and each file is about 30-50 lines. It is easy to read, change and test each class alone.

```
InventoryService.java     - stock and reservations
PaymentProcessor.java     - payments and refunds
ShippingService.java      - shipments and tracking
NotificationService.java  - messages to the customer
```

3.5 Proper Error Handling

Every failure is checked and handled. If the payment is declined, the reserved stock is released (rollback), shipping is skipped, and the customer gets a failure message. If a product is out of stock, nothing is reserved and the customer is also notified.

```
boolean paid = paymentProcessor.processPayment(customer, cart.getTotalPrice());
if (!paid) {
    inventoryService.releaseStock(order.getId());   // rollback
    order.setStatus(OrderStatus.PAYMENT_FAILED);
    notificationService.sendNotification(customer,
            "Order #" + order.getId() + " failed: payment was declined.");
    return order;                                    // shipping is not called
}
```

Test results

Example output of the failed payment scenario (Test 2), where the rollback can be seen:

```
--- Placing order #2 ---
[Inventory] All products are available
[Inventory] Stock reserved for order #2
[Payment] Declined for Bob: need 2400.0, balance is 100.0
[Inventory] Stock released for order #2
[Notification] To bob@mail.com: Order #2 failed: payment was declined.
Result: PAYMENT_FAILED
```

4. Conclusion

The `OrderFacade` hides the internal work of the system: which services exist, the order in which they are called, and how the rollback is done when the stock check or the payment fails. The client sees only `placeOrder()`, `cancelOrder()`, `addProduct()` and `getOrderStatus()`.

The subsystem classes (`InventoryService`, `PaymentProcessor`, `ShippingService`, `NotificationService`) are still public classes, so an advanced client can use them directly without the Facade, for example to read a shipment status with `ShippingService.getShipmentStatus()` or to work only with the warehouse. But then the

client is responsible for the correct order of calls and for the rollback by itself.

5. GitHub Repository Link

https://github.com/Dayglovv/SDP-Assignment-4